

Documento de Diseño y Arquitectura: Sistema Avanzado de Análisis Financiero

Este documento detalla la arquitectura, las decisiones de diseño y la sustentación técnica del proyecto de análisis bursátil, cumpliendo de manera estricta con todas las restricciones académicas y algorítmicas impuestas.

1. Arquitectura General del Sistema

El proyecto sigue una arquitectura **Monolítica Modular Basada en Datos (Data-Driven)** estructurada en tres etapas principales:

1. **Capa ETL (Extracción, Transformación y Carga):** Orquestada de forma secuencial fuera del runtime de la aplicación web. Extrae datos primarios desde APIs sin usar wrappers, los unifica temporalmente y genera un repositorio consolidado estático (`master_dataset.csv`).
2. **Capa Matemática/Algorítmica (`analysis/`):** Una librería de módulos puros en Python separada por dominio de conocimiento (Machine Learning, Similitud de Series y Teoría de Riesgo).
3. **Capa de Presentación Web (`app.py`):** Un front-end interactivo desplegado bajo Streamlit y renderizado en Canvas vectorial (Plotly), que inyecta resultados numéricos crudos a componentes visuales (HUD).

Diagrama de Flujo (Lógico)

1. `etl/extractor.py` (Módulo HTTP Requests) -> `data/raw/*.json`
 2. `etl/transformer.py` (Módulo de Sanamiento LOCF) -> `data/clean/*.csv`
 3. `etl/loader.py` (Módulo de Unificación Temporal) -> `data/master/master_dataset.csv`
 4. `app.py` -> `analysis/*.py` -> Métricas en Vivo / Gráficos / Reporte PDF
-

2. Sustentación de Requerimientos Técnicos

Requerimiento 1: Pipelines ETL Desde Cero

- **Extracción:** Se utiliza la librería estándar `requests` para invocar manualmente la API pública de *Yahoo Finance* iterando una solicitud por cada Ticker definido en `config.py` (más de 20 activos, incluyendo ADRs globales, índices S&P y mercado colombiano BVC). Se descartó explícitamente el uso de `yfinance`.
- **Transformación y Consistencia:** Para manejar los valores faltantes o nulos (*Missing Values*), se implementaron comprensiones de lista y ciclos de acarreo conocidos como LOCF (*Last Observation Carried Forward*). Esto es matemáticamente neutro, ya que asume que ante un feriado cambiario o desalineamiento de calendarios, el precio representativo del mercado sigue siendo el equivalente al último cierre válido negociado.
- **Carga:** Se usó un modelo explícito de *Outer Join Algorítmico* con diccionarios en `loader.py` para sincronizar las series colombianas con las de *Wall Street*, evitando de

manera puritana la dependencia de `.merge()` en *Pandas*.

Requerimiento 2: Algoritmos de Similitud de Series (Puro $O(N)$)

Nos desvinculamos completamente de *SciPy* y *Scikit-Learn*. Construimos explícitamente 4 algoritmos:

1. **Euclidiana:** Mide la distancia geométrica pura de ambas series normalizadas con Z-Score.
2. **Pearson:** Evalúa formalmente la covarianza transversal de dos flujos dividido por el producto cruzado de su desviación estándar.
3. **Coseno:** Determina el empuje direccional de las trayectorias de precios modeladas como trayectorias de un Vector N-Dimensional.
4. **Dynamic Time Warping (DTW):** Implementado con Programación Dinámica matricial en Listas de Listas, calculando un costo de alineación temporal acoplado elásticamente para detectar parecidos entre instrumentos bursátiles que cayeron en distintas jornadas. *Nota: La UI fue mejorada para exponer dinámicamente tanto la formulación matemática $\LaTeX$, como el costo de Big-O computacional del algoritmo escogido por el usuario en tiempo real.*

Requerimiento 3: Clasificación de Riesgo y Dispersión de Patrones

1. **Volatilidad y Riesgo (`risk.py`):** Utilizando iteradores nativos $O(N)$, se recorre el Master Dataset para obtener retornos logarítmicos $\ln(\frac{P_t}{P_{t-1}})$. Finalmente, se obtiene la Desviación Estándar muestral y se clasifica escalondamente al perfil: Conservador ($v < 0.15$), Moderado ($0.15 - 0.25$) o Agresivo ($v > 0.25$).
2. **Frecuencias Periódicas (`patterns.py`):** Mediante la técnica computacional *Sliding Window*, se desplaza un puntero para evaluar cuántas iteraciones se rompen cierres alcistas al hilo, y un validador adicional explora formaciones de *Morning Star* / *Reversión en V* cotejando las aperturas direccionales con velas anteriores completas.

Requerimiento 4 y 5: Analítica Gráfica y Dashboard

- **UI Streamlit Integrada:** A través de un contenedor único basado en tabs, se dibujan Heatmaps (iterando una matriz de covarianzas $O(C^2)$) y gráficas *Candlesticks* para las que se calculó algorítmicamente y a mano el Trazo de Fibonacci a partir de las barreras máximas y mínimas proyectadas.
- **Fuerza Bruta & Simulaciones Markowitz:** Todo es simulado calculando retornos masivos usando random walks puros (`random.Gauss` y Matrices Nativas), respetando que los estimadores Monte Carlo son elaborados por el estudiante y no pre-ensamblados de simuladores de alto nivel.
- **Documentación Autónoma y Reproducibilidad:** Todo el sistema se descarga, compila y limpia para cualquier máquina host corriendo un único comando de servidor Python.

□ 3. Declaración Explícita de Uso de Inteligencia Artificial

Para el desarrollo del presente proyecto, se utilizaron técnicas de Inteligencia Artificial Generativa bajo el modelo de *Pares de Programación (Pair-Programming)*.

1. **Soporte en Lógica Estructural Pura:** Se usó IA para validar la eficiencia

computacional (Notación Big-O) de los algoritmos propios escritos en Python.

Específicamente, para asegurarse que los iteradores manuales no tuvieran complejidad $O(N^2)$ innecesaria salvo algoritmos estrictos como el DTW.

2. **Asistencia CSS/Diseño Web:** Se usó para generar y balancear la estética Cyberpunk/Futurista basada en gradientes HEX implementable sobre el DOM de los *Streamlit Metrics*.
3. **Mapeo Vectorizado de Fórmulas:** Se asistió del LLM para transcribir las ecuaciones teóricas de la correlación de Pearson y Similitud de Coseno a convenciones de código LaTeX, a fin de incluirlas en el visualizador interactivo exigido en la rúbrica.

La herramienta actuó como tutor linter y auditor algorítmico, pero todos los *Loops*, validaciones matemáticas de similitud y heurísticas ETL fueron construidos explícita y algorítmicamente siguiendo la base normativa del taller.